

# From Linear Approximation to Deep Q-Networks

Replay, target networks, and a stable training dataflow

# Learning objectives

---

By the end of this lesson, you should be able to:

- 01 — Map a discrete-action state to one Q value per action.
- 02 — Explain temporal correlation and moving targets.
- 03 — Describe replay-buffer sampling and data reuse.
- 04 — Distinguish online and target network roles.
- 05 — Compute a terminal-aware mini-batch target.
- 06 — Trace the complete DQN training loop.

# DQN is Q-learning with engineered stabilization

---

**A neural network supplies the action-value function while replay and a target network reduce two major sources of instability.**

- One forward pass outputs all discrete-action values.
- Replay breaks short-range temporal correlation.
- A frozen target network supplies short-window consistency.

## Stabilization

The mechanisms improve the training distribution and slow movement of the bootstrap target.

# The DQN target and loss

---

$$y = r + \gamma(1 - \text{done}) \max_a Q(s_{\text{next}}, a; \theta_{\text{minus}}); L = \text{mean}[(y - Q(s, a; \theta))^2]$$

- $\theta$  are online-network parameters.
- $\theta_{\text{minus}}$  are target-network parameters.
- `done` masks every terminal bootstrap.
- Only  $\theta$  receives the gradient step.

## ENGINEERING IMPLICATION

**The two networks have distinct responsibilities even when their architectures match.**

# Online and target networks form a controlled handoff

---

## Online network

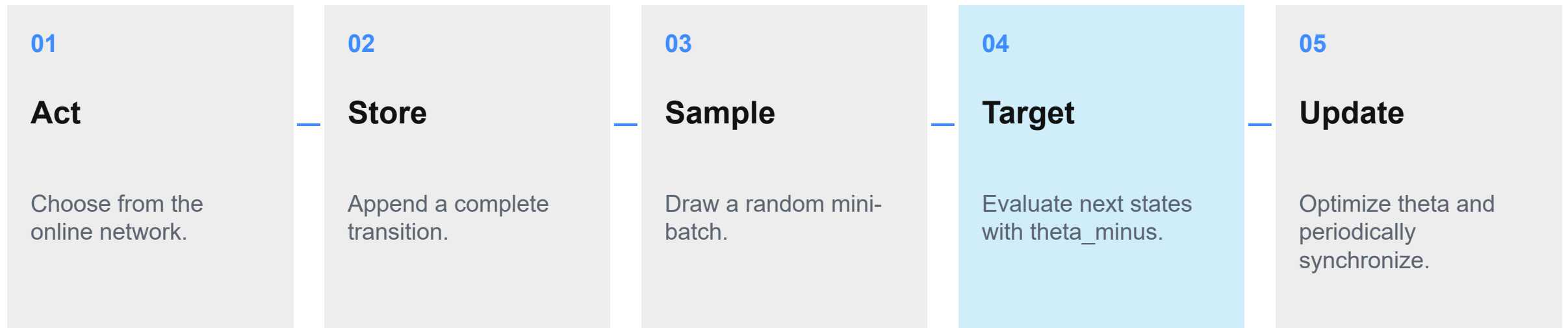
- Selects epsilon-greedy actions.
- Predicts  $Q(s,a;\theta)$ .
- Updated every training step.
- Receives gradients from loss.

## Target network

- Computes next-state target values.
- Frozen for a short interval.
- Copied or softly updated.
- Never optimized directly.

# Data flows through six explicit stages

---



Keeping action selection, target construction, and optimization separate prevents accidental gradient paths.

# High-level DQN training loop

---

```
initialize online Q(theta), target Q(theta_minus)
theta_minus = theta
replay = bounded buffer
for each environment step:
    a = epsilon_greedy(Q(s, ., theta))
    s_next, r, done = env.step(a)
    replay.add(s, a, r, s_next, done)
    batch = replay.sample(B)
    y = r + gamma*(1-done)*max_a Q(s_next,a,theta_minus)
    minimize mean((y - Q(s,a,theta))**2) over theta
    if step % C == 0: theta_minus = theta
```

## Implementation contract

- Warm up the buffer.
- Sample without sequence bias.
- Detach targets.
- Sync on a fixed schedule.

# Worked target: terminal masks matter

## SCENARIO

**A batch item has reward 2, gamma 0.9, target-network next values [1,4,3], and done=0.**

- The maximum next value is 4.
- The bootstrap contribution is  $0.9(4)=3.6$ .
- The target is 5.6.
- If done becomes 1, the multiplicative mask removes 3.6.

**The nonterminal target is 5.6; the terminal target is exactly 2.**

# Three training regimes expose the design

---

## Naive online

Newest sample

- Strong correlation.
- No data reuse.
- Same moving network.
- Often oscillatory.

## Replay only

Random batches

- Better sample mixture.
- Reuses experience.
- Target still moves fast.
- Needs buffer policy.

## Full DQN

Replay plus target

- Decorrelated batches.
- Short-window targets.
- More state to manage.
- Still no guarantee.

# Checkpoint: find the forbidden gradient

---

## CHECKPOINT

**A student computes  $y$  with the online network and allows automatic differentiation through  $y$ . What is wrong?**

---

01

Identify both target defects.

02

Name the parameter set that should update.

03

Describe the required stop-gradient.

# Replay changes the training distribution

---

## Sequential stream

- Adjacent states are similar.
- Recent policy dominates.
- One transition used once.
- Ordering can bias gradients.

## Random mini-batch

- Samples mix times and episodes.
- Older policies remain represented.
- Transitions are reused.
- Distribution still reflects buffer contents.

# Hard and soft target updates

---

hard:  $\theta_{\text{minus}} \leftarrow \theta$  every  $C$  steps; soft:  $\theta_{\text{minus}} \leftarrow \tau \theta + (1-\tau) \theta_{\text{minus}}$

- $C$  is the hard-copy interval.
- $\tau$  is a small interpolation rate.
- Hard updates create piecewise-constant targets.
- Soft updates move every step but more slowly.

## ENGINEERING IMPLICATION

**The synchronization rule is a hyperparameter and must be logged with results.**

# Replay sampling breaks a correlated run

---

## SCENARIO

**The last eight transitions all come from the same corridor, while a 10,000-item buffer spans many states.**

- Online-only training sees eight nearly identical inputs.
- A uniform batch draws across older episodes.
- Gradient directions reflect several regions.
- Repeated data use improves sample efficiency.

**Replay reduces immediate correlation but cannot repair a buffer that never contains rare successful transitions.**

# Buffer choices create different evidence

---

## Capacity

How much history?

- Small adapts quickly.
- Large increases diversity.
- Large can retain stale data.
- Memory cost is explicit.

## Sampling

Which transitions?

- Uniform is simple.
- Prioritized focuses error.
- Weights may be needed.
- Seed controls reproducibility.

## Warm-up

When train starts

- Avoid tiny biased batches.
- Delay early gradients.
- Costs interaction steps.
- Log threshold.

# Checkpoint: remove one stabilizer

---

## CHECKPOINT

**What symptom would you expect without replay, and what different symptom would you expect without a target network?**

---

01

Separate data correlation from target motion.

02

Name an observable metric.

03

Avoid claiming a theoretical cure.

# DQN adds state that must be checkpointed

---

**A reproducible DQN run includes more than neural-network weights.**

- Serialize online and target networks separately.
- Record buffer capacity, sampling seed, and sync cadence.
- Evaluate with frozen parameters and epsilon zero.

## State

Replay contents, target parameters, optimizer moments, counters, epsilon, and random states affect continuation.

# Key takeaways

---

- 01 — DQN maps one state to all discrete-action values.
- 02 — Replay reduces temporal correlation and reuses data.
- 03 — Target networks slow bootstrap-target movement.
- 04 — Both mechanisms mitigate, but do not eliminate, deadly-triad risk.

Next: Bayesian networks introduce a different way to represent uncertainty: explicit probabilistic structure.

# Check your understanding

---

Q1

Compute a masked DQN target.

Q2

Explain why  $y$  must be detached.

Q3

Compare hard and soft synchronization.

Q4

Describe replay warm-up.

Q5

Name all checkpoint state.